PRISMA SaaS Platform

TECHNICAL SPECIFICATION MANUAL

Architecture, System Flows, and Onboarding Journeys

Document Type: *Product Architecture and Flow Specification*
Version: *1.0 (Enterprise Release)*
Date: *June 2026*
Security: *Restricted / Confidential*
Target Audience: *Engineering, Product Management, Security, and Customer Success Teams*

1. System Architecture Overview

PRISMA is a vertically-integrated, multi-tenant Software-as-a-Service (SaaS) platform designed to deliver predictive forecasting and supply chain optimization across multiple industry verticals, specifically **Apparel & Fashion**, **Consumer Electronics**, and **Pharmaceuticals** (compliant with GxP and 21 CFR Part 11).

The platform utilizes a modern containerized micro-frontend and API architecture backed by PostgreSQL with strict Row-Level Security (RLS) for tenant isolation. The diagram below and subsequent tables describe the high-level layout of the application components:

Component	Technology Stack	Core Responsibility
Frontend SPA	Vite + React + TypeScript Nginx (Port 8080)	Single-page app displaying real-time supply chain metrics, demand forecasts, GxP audits, and tenant configurations.
Backend API	FastAPI (Python 3.11+) Uvicorn (Port 8000)	Handles auth (JWT/Firebase), workspace routing, metadata CRUD, audit logging, and logical tenant separation middleware.
ML Inference API	FastAPI (Python) Ensemble + Chronos (Port 8001)	Serves predictive forecasts using pre-trained stacked ensembles or falls back to Amazon Chronos Zero-Shot deep learning models.
Database	PostgreSQL 16 + pgvector Strict Row-Level Security (RLS)	Contains shared relational schemas. Tenant isolation is enforced at the DB-connection level using tenant UUIDs.

1.1 Multi-Tenant Row-Level Security (RLS)

To ensure complete logical isolation between enterprise customers (tenants):

- **Tenant Boundary:** Every database table containing customer-owned data includes a tenant_id UUID column.
- **Connection Setting:** For every incoming API request, the FastAPI database session helper issues a SET LOCAL app.current_tenant_id = '<uuid>' command.
- **Database Enforcement:** PostgreSQL Row-Level Security (RLS) policies block any read, update, or delete operations that do not match the current session's tenant ID. This prevents cross-tenant data leaks at the query execution level.

2. Onboarding & Authentication Journeys

This section details the explicit user journeys and backend mechanics for the three entry points: **Self-Service Sign-Up**, **Demo Sandbox**, and **Standard Sign-In**.

2.1 Journey A: Self-Service Sign-Up

This flow allows a new organization to establish a brand-new tenant account. Self-service sign-up strictly prevents users from joining existing tenant workspaces to avoid unauthorized entry into another customer's data.

Step	Action & Backend Logic
1. Input	User goes to '/login?mode=signup' and inputs their Full Name, Email, Password, and a desired Tenant Slug (e.g., 'acme-pharma').
2. Request	Frontend sends a POST request to '/api/v1/auth/signup' containing registration details.
3. Validation	Backend performs checks: - Email is checked for uniqueness in the database. - Tenant Slug is checked to ensure no tenant exists with that name. If it exists, a ConflictError is thrown.
4. DB Provisioning	The backend creates: - A new Tenant record (status: Trial, tier: Growth) with allowed verticals. - Default IndustryProfile configurations (enabling vertical-specific feature flags like GxP mode for Pharma). - A new User record (role: Owner) with password hashed using Argon2id.
5. Firebase Sync	If Firebase is enabled, backend creates the user in Firebase Auth and immediately assigns custom tenant claims: tid (Tenant ID), puid (User ID), role (owner), ind (active industry).
6. Response	The backend returns an 'ok' status, redirecting the user to sign-in.

2.2 Journey B: 'Try the Sandbox' (Public Demo Access)

To let prospective clients evaluate PRISMA without registering, the 'Try the Sandbox' flow authenticates them into a pre-seeded public tenant account. Crucially, this happens passwordlessly and entirely server-side so no credentials are ever exposed in client-side bundles.

Step	Action & Backend Logic
1. UI Click	User clicks 'Try the Sandbox' on the marketing page.
2. Session Call	Frontend sends a empty-body POST to '/api/v1/auth/sandbox-session'.
3. Tenant Resolution	Backend checks configurations to find the configured sandbox tenant slug ('prisma-dev') and sandbox email ('owner@prisma-dev.com').

4. Token Minting	The backend creates a custom auth token representing the demo tenant/owner user. The token encodes custom tenant claims (tid, puid, role, active_industry).
5. Audit Log	Backend commits a 'user.sandbox_login' event in the central database audit table (recording timestamp, IP, and user-agent).
6. Session Start	The backend returns the SandboxSessionResponse containing the JWT/Firebase custom token. The React SPA stores it and authenticates the user instantly, redirecting them to the live dashboard.

2.3 Journey C: Standard Sign-In & API Security

Once a user has an active account, standard sign-in authenticates them and injects tenant-context claims into all subsequent API traffic.

1. Credentials Check: The user enters their email and password. The backend verifies the password hash (Argon2id) or validates a Google OAuth ID token.
2. JWT Generation: The backend issues a JWT containing custom claims: tid (Tenant ID UUID), role (e.g. Owner, Analyst), ind (Active Industry), and industries (Licensed verticals). Access tokens expire in 60 minutes; refresh tokens are stored as SHA-256 hashes for security.
3. Client-Side Storage: The Vite + React SPA receives the token, stores it in useAuthStore, and attaches it as a Bearer token in the Authorization header of every Axios API request.
4. TenantContextMiddleware: For every request, backend middleware intercepts the token, extracts the claims, verifies signature, and writes details to request.state. When query runs, the middleware applies the tenant filter to the database session, enforcing strict RLS.

Step 1: Sign-In Page	Step 2: Backend API	Step 3: Database Session
User enters password. Client POSTs to `/auth/login`.	Verifies hash, issues JWT with tenant ID (`tid`) & role claims.	Saves login event to append-only `audit_log`.
Step 4: SPA Storage	**Step 5: Middleware**	**Step 6: RLS Enforcement**
SPA stores JWT, appends it as `Authorization: Bearer ` to requests.	`TenantContextMiddleware` intercepts, parses `tid`, and binds it to database session.	`SET LOCAL app.current_tenant_id` blocks any SQL commands targeting other tenants.

3. After Sign-In: What the User Gets

Upon successful authentication (either standard or via the sandbox), the user is redirected to the main Dashboard. What the user sees and is authorized to do depends on their active industry vertical, role permissions, and tenant tier.

3.1 Vertical Industry Dashboards

PRISMA dynamically adapts its user interface based on the active industry. Users can toggle between industries via a selector in the header (which overrides the default backend vertical by sending the X-Industry-Code header):

Vertical	Key UI Modules & Capabilities Provided

Apparel & Fashion	• **Style Demand Forecasting:** Forecasts styles with short lifecycles, seasonal trends, and high product variety. • **Assortment Planning:** Guides store-level distribution, color, and size breakdown based on historic sales profiles. • **Markdown Optimizer:** Simulates price-decay strategies to clear seasonal stock while maximizing margins.
Consumer Electronics	• **Lifecycle Sales Forecasting:** Predicts demand curves for new launches, incorporating obsolescence profiles. • **Component Risk Tracker:** Evaluates raw component supply dependencies and tracks vendor risk scores. • **Competitor Tracker:** Visualizes real-time competitor prices scraped from online marketplaces.
Pharmaceuticals	• **GxP Batch Compliance:** Displays batch numbers, manufacturing dates, and cold-chain temperature logs. Requires strict QA-release approvals before dispatching forecasts. • **Drug Shortage Predictor:** Identifies API (Active Pharmaceutical Ingredient) bottlenecks and predicts risk of regulatory stockouts. • **Immutable Audit Log:** Lists every user action in an append-only ledger, compliant with 21 CFR Part 11 regulations.

3.2 Predictive Forecasting Engine

Users get access to a powerful demand forecasting pipeline. When a user requests a forecast (via the UI or API), PRISMA dynamically routes the demand series:

1. **Trained Ensemble Path:** If the tenant has seeded data and completed a training pass, the platform loads the pre-trained StackedEnsemble model bundle (combining statistical models, GBT, deep neural nets, and foundation forecasters) to generate precise forecasts optimized for their historical patterns.

2. **Zero-Shot Fallback:** During onboarding or right after signup, when no custom models are trained yet, the backend transparently routes requests to a ZeroShotForecaster using Amazon's **Chronos zero-shot deep learning model**. This ensures the user gets valuable predictions instantly without needing prior model training.

3.3 Supply Chain & Inventory Optimization

PRISMA translates demand forecasts directly into actionable logistics advice. The Inventory Optimization module provides:
• **Safety Stock Calculator:** Calculates optimal stock buffers considering vendor lead-time variability and target service levels (e.g., 95% or 99% service rate).
• **Reorder Point (ROP) Alerts:** Alerts procurement managers when warehouse levels cross the reorder threshold.
• **Economic Order Quantity (EOQ):** Optimizes order sizes to balance holding costs and order-setup expenses.

3.4 Role-Based Access Control (RBAC)

The dashboard restricts capabilities based on the user's role:

Role	Privileges & Dashboard Permissions
Owner / Admin	Full access. Can configure tenant settings, manage licenses, invite/delete team members, upload master datasets, override model weights, and sign off on forecasting releases.
Analyst	Can run forecasts, generate scenarios, override promotional and discount parameters, trigger model retrain requests, and download CSV reports. Cannot modify team configurations.
Viewer	Read-only access. Can view dashboard graphs, track forecasts, and read audit logs, but cannot edit settings, trigger forecasts, or modify data.